

JIBBLE 30-DAY DEVELOPMENT PLAN

=====

Goal: Build the first functional version of the Jibble app (backend + Flutter mobile + basic infra + admin panel v1) in 30 days, following the plan below.

Assumptions

- Solo developer (you), 5–8 focused hours per day.
- Tech stack: Node.js + Express + PostgreSQL (Neon), Redis, S3/Cloudinary, Socket.io, Flutter (Riverpod or similar), basic CI/CD.
- You already have VS Code, Flutter SDK, Node, Git, and GitHub set up.
- Scope: MVP-quality implementation of the core features from the PRD + minimal admin panel (just enough for seeding/management).

Legend

- BE = Backend
- FE = Flutter mobile app
- ADM = Admin panel (web)
- INF = Infrastructure / DevOps
- DOC = Documentation

Week 1 – Project Setup, Auth, and Core Foundations (Days 1–7)

Day 1 – Monorepo & Repo Setup

- Create monorepo root `jibble/` with `apps/`, `backend/`, `infrastructure/`, `database/`, `shared/`, `docs/`.
- Initialize Git repository and create main branches (`main`, `dev`).
- Add base `README.md` describing project, tech stack, and folder structure.
- Setup backend Node project (package.json, tsconfig, basic `src/index.ts`, `src/app.ts`).
- Setup Flutter project under `apps/mobile\_app/` (run `flutter create`).

Day 2 – Backend Core Config & Env

- Implement `backend/src/config/env.ts` with dotenv + schema validation.
- Implement `db.ts`, `redis.ts`, `s3.ts`, `cloudfront.ts`, `logger.ts`, `requestContext.ts`.
- Implement middlewares: `correlation.middleware.ts`, `requestLogger.middleware.ts`, `error.middleware.ts`.
- Add `api/v1` router entry and mount a health-check route (`GET /api/v1/health`).
- Create `.env.example` with all required keys.

Day 3 – Database Schema & Migrations

- Design/confirm core tables in `database/schema.sql` (users, profiles, colleges, posts, stories, comments, likes, follows, messages, events, ...).
- Implement SQL migrations in `backend/src/db/migrations/\*`.
- Implement migration scripts (`infrastructure/scripts/migrate\_db.sh`, `rollback\_db.sh`).
- Run migrations against local/Neon DB and verify tables.

Day 4 – Auth Module (BE) – Entities & Repositories

- Create `modules/auth/` and `modules/user/` folders.
- Define `user` entity in ORM/schema and `user.types.ts`.
- Implement `user.repository.ts` (create user, find by id/email/username, update basic fields).
- Implement `auth.repository.ts` for tokens/sessions if needed.
- Add `auth.validation.ts` (signup, login, refresh, reset password).

Day 5 – Auth Module (BE) – Services & Controllers

- Implement `auth.service.ts` (signup, login with password, token generation, refresh, logout).
- Implement `auth.controller.ts` + `auth.routes.ts` under `api/v1/auth`.
- Wire in rate limiting for auth (`rateLimiter.auth.ts`).
- Add unit tests for core auth flows in `tests/auth.test.ts`.

Day 6 – Flutter Setup & Auth Screens (FE)

- Setup base Flutter architecture: `core/`, `features/`, `shared/` as per blueprint.
- Implement `core/network/api\_client.dart`, `endpoints.dart`, `network\_interceptor.dart`.
- Build UI for `splash\_screen.dart`, `login\_page.dart`, `register\_page.dart`, `otp\_verification\_page.dart` (if used).
- Setup state management for auth (`auth\_provider.dart`).

Day 7 – Connect Auth (FE ↔ BE)

- Integrate auth API calls in ``auth_service.dart`` (signup, login, logout, get current user).
- Implement persistent auth state (store tokens securely, auto-login if valid).
- Implement ``auth_gate.dart`` to redirect to onboarding/home based on login state.
- Manual end-to-end test: create account, login, see basic home placeholder.

Week 2 – Onboarding, Profiles, Feed, and Posts (Days 8–14)

Day 8 – Profile & College (BE)

- Implement ``college`` module (controller, service, repository, routes) for listing/selecting colleges.
- Implement ``profile`` module with profile fields from PRD (name, username, dob, interests, skills, etc.).
- Add endpoints for updating profile and fetching own profile.
- Seed ``colleges`` table with ~20 initial entries using ``seed_colleges.ts``.

Day 9 – Onboarding Flow (FE)

- Implement onboarding screens: ``college_page.dart``, ``date_of_birth_page.dart``, ``interests_page.dart``, ``skills_page.dart``, ``profile_picture_page.dart``.
- Connect these screens to BE endpoints (colleges list, profile update, file upload for profile picture).
- Add validation and local error handling.

Day 10 – Feed & Post Entities (BE)

- Implement ``post`` module entities and DB relations (posts, likes, comments, categories).
- Implement ``post.repository.ts`` with queries for:
 - create post
 - get home feed (paginated)
 - get user posts
 - toggle like
- Implement ``post.service.ts`` with business rules (visibility, max length, etc.).

Day 11 – Feed & Post APIs (BE)

- Implement ``post.controller.ts`` + ``post.routes.ts`` under ``/api/v1/posts``.
- Add validation for create/delete/toggle-like, pagination.
- Add rate limiting for posting (``rateLimiter.post.ts``).
- Basic tests for post creation and feed retrieval.

Day 12 – Home Feed UI (FE)

- Implement ``home_page.dart`` layout (stories row, feed list, FAB or create button).
- Implement ``feed_post_card.dart``, ``stories_row.dart``, ``category_chips_row.dart``.
- Connect ``home_service.dart`` to ``/api/v1/posts`` feed endpoints.
- Wire ``feed_provider.dart`` to load paginated feed.

Day 13 – Create Post UI (FE)

- Implement ``create_post_page.dart`` (text + media attachments).
- Integrate media upload to S3/Cloudinary via BE upload endpoint.
- Implement optimistic UI updates for likes and new posts.

Day 14 – Stories (BE + FE)

- Implement ``story`` module (BE: controller, service, repository, routes).
- Implement basic story UI: ``stories_provider.dart``, ``story_viewer_page.dart``, ``story_bubble_widget.dart``.
- End-to-end tests: create story from app, visible on home.

Week 3 – Circle, Chat, Notifications, and Search (Days 15–21)

Day 15 – Circle Model & Components (BE)

- Implement ``circle`` module: community membership, circle configuration, component toggles.
- Implement ``community_components`` and ``component_templates`` tables.
- Add endpoints to fetch circle config for a user (which tabs/features enabled).

Day 16 – Circle UI Shell (FE)

- Implement ``circle_page.dart`` as a tabbed interface (Anonymous, Events, Groups, Matching) based on config from BE.
- Implement navigation from bottom nav bar to Circle.
- Load circle config on app start and cache it.

Day 17 – Anonymous & Events (BE + FE)

- Anonymous:
 - BE: ``circle.anonymous.service.ts``, repository for anonymous posts.
 - FE: ``anonymous_tab_page.dart``, list + create anonymous post.
- Events:
 - BE: event creation/listing for a circle.
 - FE: ``events_tab_page.dart``, ``circle_event_card.dart``.

Day 18 – Groups & Matching (BE + FE)

- Groups:
 - BE: groups, group_members tables and module.
 - FE: ``groups_tab_page.dart``, ``circle_group_tile.dart``, create/join flow.
- Matching:
 - BE: basic matching cards and simple algorithm.
 - FE: ``matching_tab_page.dart``, simple swipe or list-based UI.

Day 19 – Chat (BE Socket + REST)

- Implement ``chat`` module with:
 - ``chat.repository.ts`` for conversations and messages.
 - ``chat.service.ts`` business logic.
 - ``chat.socket.ts`` Socket.io events for real-time messaging.
 - REST endpoints for recent chats and loading older messages.
- Add per-module rate limiter ``rateLimiter.chat.ts``.

Day 20 – Chat UI (FE)

- Implement ``chat_list_page.dart``, ``chat_arena_page.dart``, ``message_bubble_widget.dart``.
- Implement Socket.io integration in Flutter (connect, join rooms, send/receive messages).
- Implement simple typing indicator or online status later if time allows.

Day 21 – Notifications & Basic Search

- Notifications:
 - BE: ``notification`` module (repository, service, controller) + queue integration ``notification.queue.ts``.
 - FE: ``notifications_page.dart``, ``notification_tile.dart``, FCM/local notifications wiring.
- Search:
 - BE: ``search`` module using PostgreSQL full-text search (users, groups, events).
 - FE: ``search_page.dart``, ``search_result_tile.dart`` with users/groups/events tabs.

Week 4 – Admin Panel, Analytics, Hardening, and Polish (Days 22–30)

Day 22 – Queues, Workers, and Email

- Implement ``queues/queues.config.ts`` and initialize BullMQ / Redis queues.
- Implement ``email.queue.ts`` and simple email sender for verification/reset (can be mocked initially).
- Implement ``worker.ts`` to process email and notification jobs.
- Update relevant services (auth, notifications, events) to enqueue jobs, not send synchronously.

Day 23 – Analytics Module (BE)

- Implement ``analytics`` module (repository, service, controller) for DAU/MAU, active communities, basic engagement metrics.
- Add scheduled jobs via ``analytics.queue.ts`` or cron (depending on infra choice).
- Expose ``/api/v1/analytics`` endpoints for admin panel.

Day 24 – Admin Panel (ADM) – Basics

- In ``apps/admin_panel/``, bootstrap a simple React/TypeScript app if not already.
- Implement login screen for admin panel (can reuse auth endpoints with role checks).
- Implement layout (sidebar: Dashboard, Admins, Communities, Analytics).

Day 25 – Admin Panel – Admin & Community Management

- Build Admins screen:
 - List admins with role and status.
 - Form to invite/create new admin (Head Admin only).
- Build Communities screen:
 - List communities.
 - Create/edit community forms.

- Attach/detach Circle components via UI toggles.

Day 26 – Admin Panel – Analytics & Activity Logs

- Implement Analytics overview dashboard consuming `/api/v1/analytics``.
- Implement activity log view reading from admin activity log endpoint.
- Add basic filters (date range, admin, action type).

Day 27 – Feature Flags, Logging, and Rate Limiting Review

- Implement `featureFlags.ts`` (BE) and add guards in critical modules (e.g., toggle Matching or Marketplace).
- Verify structured logging with `logger.ts`` and `requestContext.ts`` (requestId/correlationId appearing in logs).
- Tune per-module rate limits based on expected usage patterns.

Day 28 – Hardening: Validation, Errors, and Security

- Review all endpoints for input validation (zod/joi schemas).
- Make sure all errors go through `error.middleware.ts`` with consistent structure.
- Verify auth + RBAC on protected routes (Circle, Chat, Admin, etc.).
- Double-check CORS, HTTPS assumptions, and secure headers via Nginx or Express.

Day 29 – End-to-End Testing & Bug Fixes

- Run manual flows on a real device/emulator:
 - Sign up → Onboarding → Home → Feed → Circle tabs → Chat → Notifications.
 - Try from at least 2 different test users.
- Fix high-priority bugs found during flows.
- Do quick performance check (basic profiling of slow endpoints).

Day 30 – Cleanup, Documentation, and Launch Prep

- Update `docs/architecture.md`` and `docs/api-design.md`` to reflect the final implementation.
- Write a short deployment playbook (how to deploy backend, run migrations, seed data).
- Ensure `.env.example`` is fully updated.
- Clean up TODO comments and dead code.
- Tag a `v0.1.0`` release in Git and write release notes.

Notes

- If any day slips, use buffer on Days 29–30 to catch up.
- Prioritize: Auth + Onboarding + Feed + Circle (+ Anonymous) + Chat as absolute must-have features.
- Admin panel can remain minimal in v0.1: focus on admin login, communities list, and basic analytics.